

# Agentic LLM 기반 PleOS IVI APK 보안 취약점 탐색 및 검증 방법 연구

---

Agentic LLM을 이용하여 취약점 후보 탐지와 Android 검증을 통한 보안 이슈 선별

김호연 · 2025311610  
자율주행연구프로젝트1

# 목차

연구 배경에서 시작해 분석 방법, 검증 결과, 대표 사례, PoC, 개선안 순서로 설명한다.

| 순서 | 장             | 핵심 내용                                                      |
|----|---------------|------------------------------------------------------------|
| 1  | 연구 개요         | 왜 이 연구를 했고, 무엇을 산출했는가                                      |
| 2  | 위협 모델 및 분석 범위 | 어떤 APK를 보안 분석 대상으로 삼았는가                                    |
| 3  | 분석 파이프라인      | Agentic LLM을 이용하여 취약점 후보 탐지와 Android 검증을 어떻게 결합했는가         |
| 4  | 전체 분석 결과      | 취약점 후보 47건을 어떻게 탐지하였고 어떤 것들이 있는가                           |
| 5  | 주요 검증 사례      | 취약점이 Android Provider, Transport, Receiver와 연결되어 악용될 수 있는가 |
| 6  | PoC 및 영향 검증   | PleOS 에뮬레이터에서 확인한 실제 공격 시나리오는 무엇인가                         |
| 7  | 개선 방안 및 결론    | PleOS에 어떤 사이버보안 기술을 적용해야되는가                                |

## 1.1 연구 목표와 수행 내용

이 연구는 PleOS IVI APK의 외부 접점에서 보안 이슈 후보를 찾고, Android 권한·호출 경로 검증으로 보고 대상을 선별했다.

---

| 구분 | 내용                                                                                                                              |
|----|---------------------------------------------------------------------------------------------------------------------------------|
| 문제 | IVI APK는 차량 화면, 계정, 로그, 앱 간 통신과 연결되어 일반 앱보다 보안 경계가 복잡하다.                                                                        |
| 목표 | Agentic LLM으로 보안 이슈가 있을만한 취약점 후보를 넓게 찾고, 실제 공격이 가능한 이슈만 남긴다.                                                                    |
| 방법 | Android Manifest, Java decompiled code, Android permission 등을 확인해 취약점의 trigger 가능성을 평가한다.<br>시나리오를 설계하여 실제 공격 PoC를 만들어 이를 검증한다. |
| 결과 | Agentic LLM이 찾은 취약점 후보 47건에 대하여, 오탐 9건을 제거하였고, 대표 이슈와 사이버보안 개선안을 정리했다.                                                          |

Agentic LLM을 이용하여 취약점 후보를 넓게 찾고, Android 악성 APK 기반 시나리오를 바탕으로 악용 가능성을 검증했다.

## 1.2 연구 추진 배경

IVI APK에서는 API 코드 조각만으로 취약점을 판단하기 어렵고, 실제 외부 도달 가능성을 함께 봐야 한다.

---

**IVI APK는 차량 화면, 계정, 진단 로그, 앱 간 통신, 네트워크 전송과 연결된다.**

- 코드에 위험한 API가 있어도 외부 앱이 해당 코드에 도달할 수 없으면 실제 보안 이슈가 아닐 수 있다.
- Android manifest, permission, protected broadcast, signature-level 제한은 실제 공격 가능성을 크게 바꾼다.
- 따라서 후보 탐지와 Android 권한·호출 경로 검증을 분리해야 오탐을 줄일 수 있다.

**핵심 질문: PleOS IVI APK에서 어떤 후보를 실제 보안 이슈로 남길 수 있는가?**

## 1.3 최종 산출물

최종 산출물은 후보 목록, 검증 결과, 대표 사례, PoC 근거, 개선안이다.

| 산출물         | 구체 내용                                                                                              |
|-------------|----------------------------------------------------------------------------------------------------|
| 보안 이슈 후보 목록 | Provider, Receiver, Transport, Credential 등의 취약점 존재 47건                                            |
| 보안 이슈 검증 결과 | Agentic LLM 기반 1차 취약점 후보 탐지 결과에 대한 Android 공격 시나리오 검증                                              |
| 대표 사례       | LLM prompt provider, plaintext gRPC, VehicleBroadcastReceiver, provider query, receiver delivery 등 |
| PoC         | Navigation Map Route 화면 영향, MIRROR_FOLD 상태 값 변경 등                                                  |
| 개선안         | 권한 확인, TLS/mTLS, receiver validation, Binder caller check 등                                        |

취약점 후보에 대해서, 어떤 근거로 공격이 가능하고, 어떤 근거로 공격이 불가능한지 함께 정리했다.

## 2.1 PleOS IVI APK 공격 접점

분석 대상은 APK 내부 코드 전체가 아니라 외부 앱·시스템·네트워크와 만나는 접점이다.

| 공격 접점             | 보안상 의미                           | 확인한 내용                                     |
|-------------------|----------------------------------|--------------------------------------------|
| Provider          | 앱 내부 데이터가 외부 query로 노출될 수 있음     | exported, authority, read/write permission |
| Receiver          | 외부 broadcast 입력이 앱 동작으로 이어질 수 있음 | action, permission, payload validation     |
| Binder service    | 앱 간 직접 호출 경로가 열릴 수 있음            | caller UID, exported, permission           |
| Credential / SSO  | 계정·토큰·secret이 APK에 남을 수 있음       | hardcoded value, token pattern             |
| Network transport | 로그·진단 데이터가 평문 전송될 수 있음           | plaintext, TLS/mTLS 여부                     |
| Route / Property  | IVI 화면 또는 차량 상태 값과 연결될 수 있음      | UI 영향, property 접근 가능성                     |

## 2.2 주요 공격 접점 설명

각 접점은 “외부에서 도달 가능한가”와 “보호 규칙이 있는가”를 기준으로 판단했다.

| 접점         | 쉽게 말하면                | 최종 판단 기준                      |
|------------|-----------------------|-------------------------------|
| Provider   | 앱 내부 DB나 데이터를 제공하는 입구 | 외부 query 가능 + 권한 없음이면 위험      |
| Receiver   | broadcast 입력을 받는 컴포넌트 | 일반 앱이 action을 보낼 수 있으면 검증 필요  |
| Binder     | 앱끼리 직접 호출하는 서비스 경로    | caller 확인이 없으면 권한 우회 가능성      |
| Credential | 앱 안에 남은 비밀값           | 실제 secret/token이면 제거 또는 회전 필요 |
| Transport  | 서버와 통신하는 전송 경로        | 진단/로그 전송에 plaintext면 보호 필요    |

중요한 기준은 코드에 무엇이 있느냐보다, 외부 앱이 그 코드까지 도달할 수 있느냐다.

## 2.3 분석 자료와 환경

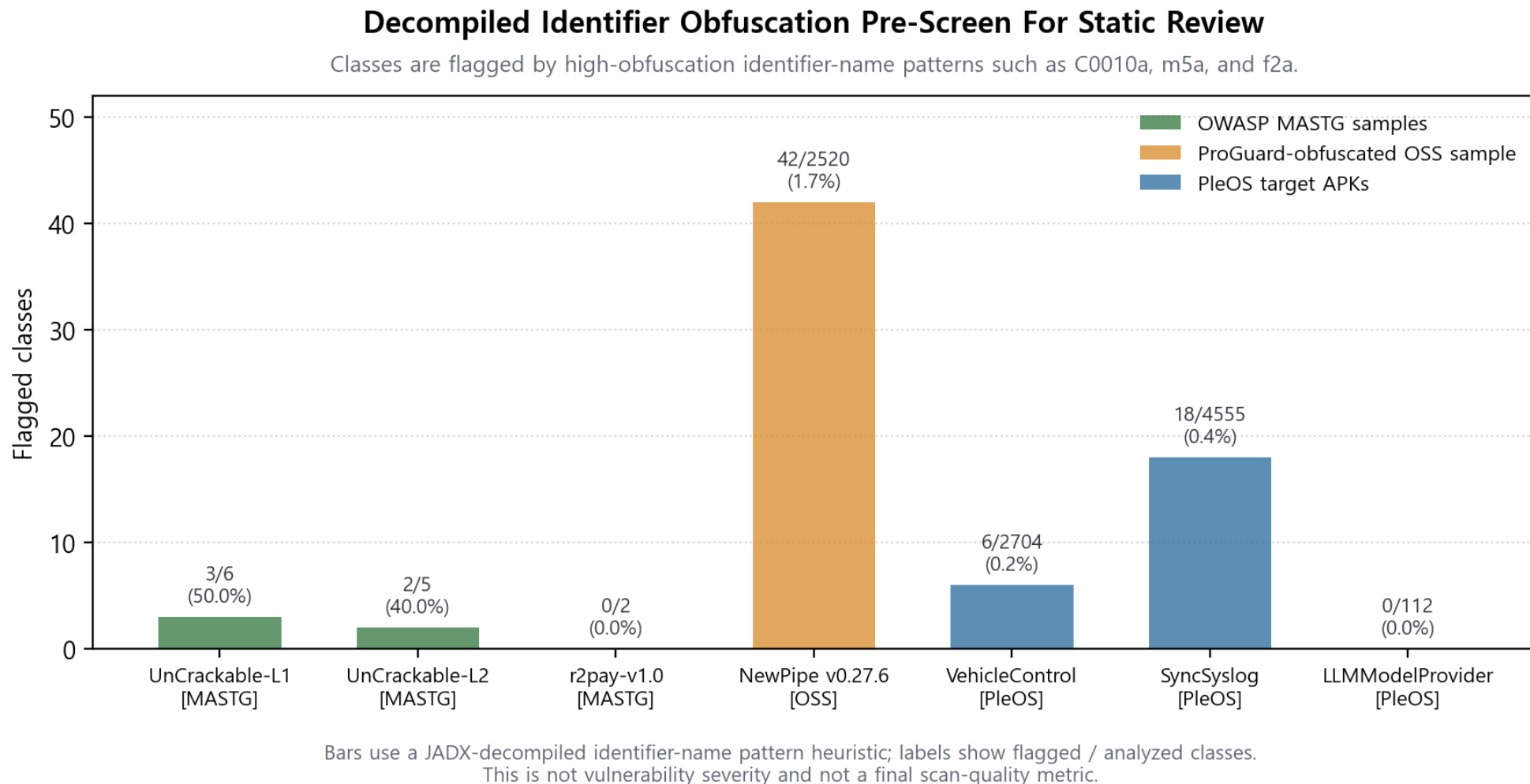
최종 판단은 manifest, decompiled code, Android 설정, 에뮬레이터 검증 결과를 함께 사용했다.

| 자료                  | 확인한 내용                                    | 판단에 쓴 이유         |
|---------------------|-------------------------------------------|------------------|
| AndroidManifest.xml | exported, permission, authority, action   | 외부 노출 여부 확인      |
| Decompiled code     | source/sink, handler, gRPC, credential    | 실제 동작 경로 확인      |
| Android/PleOS 설정    | protected broadcast, signature permission | 시스템 보호 규칙 확인     |
| Emulator PoC        | query, broadcast, property, route 영향      | 일부 경로의 호출 가능성 확인 |

예: receiver가 exported되어 있어도 protected broadcast만 받는다면 일반 앱이 임의로 트리거하기 어렵다.

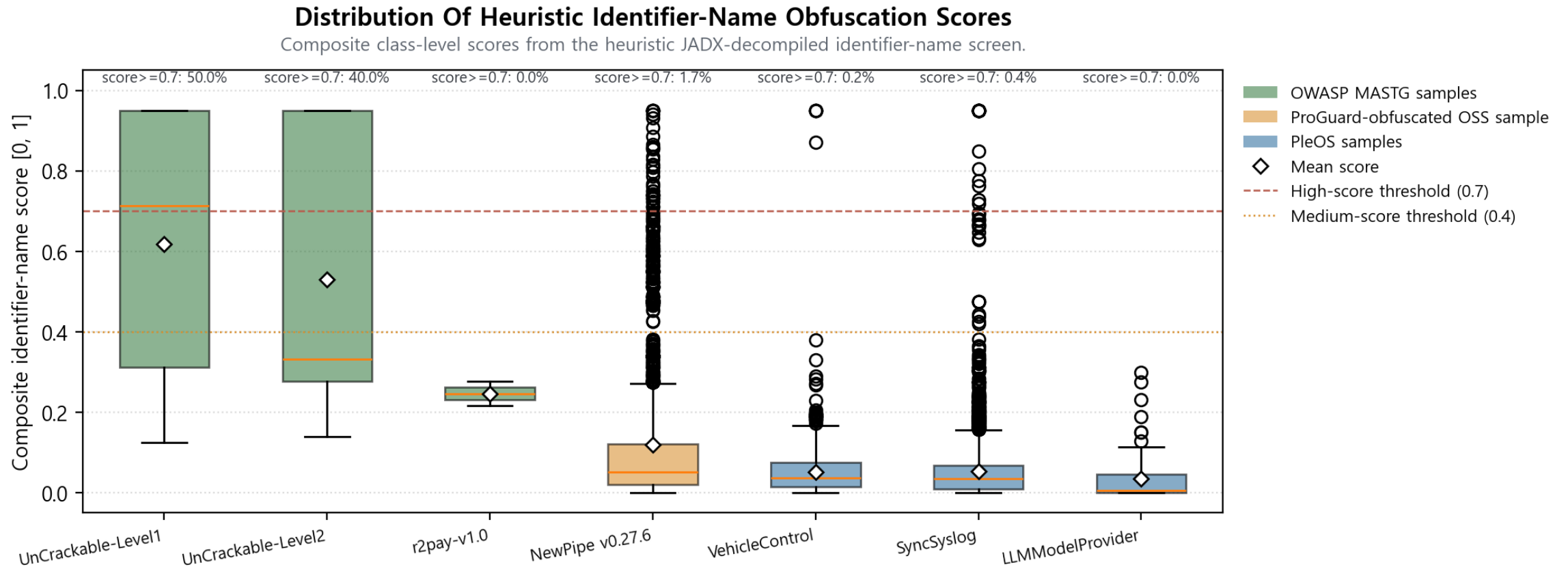
## 2.3 분석 자료와 환경

정적분석 사전 가능성 확인을 위해 난독화가 얼마나 되어 있는지 파악하였다.



## 2.3 분석 자료와 환경

난독화 점수 분포는 취약점 심각도가 아니라 정적 리뷰 난이도를 해석하기 위한 보조 근거다.



This shows the score spread behind the flagged-class screen; thresholds are heuristic readability markers, not an obfuscation benchmark or vulnerability metric. Diamonds mark mean class-level score.

## 2.4 주장 범위

이 연구의 주장은 APK 정적분석과 에뮬레이터 검증 범위로 제한한다.

| 주장하는 것                                  | 주장하지 않는 것                        |
|-----------------------------------------|----------------------------------|
| PleOS IVI OEM APK에서 확인한 보안 이슈 후보와 검증 결과 | OEM APK를 통한 실차 제어 성공             |
| Android 권한 및 Binder 등의 외부 호출 가능성 파악     | Android 원격 공격 성공                 |
| 에뮬레이터 PoC: Navigation 경로 호출 조작          | 자율주행 탈취                          |
| PleOS에 적용 가능한 보안 개선안                    | Production 수준의 end-to-end 보안 개선안 |

PoC는 정적분석 후보 중 일부에 대하여 에뮬레이터에서 공격 시나리오로 악용 가능한지 확인한 보조 근거다.

## 3.1 전체 분석 절차

전체 절차는 후보 탐지, Android 검증, 최종 이슈 선별, 개선안 도출로 구성된다.

**APK 분석 → LLM 후보 탐지 → Android 검증 → 최종 보안 이슈 선별 → 개선안 도출**

| 단계            | 입력                                   | 수행 내용               | 산출물         |
|---------------|--------------------------------------|---------------------|-------------|
| 1. APK 분석     | Manifest, decompiled code            | 접점과 위험 패턴 추출        | 분석 대상 코드 위치 |
| 2. LLM 후보 탐지  | 코드 excerpt, manifest 정보              | 보안 이슈 후보 생성         | 후보 47건      |
| 3. Android 검증 | permission, exported, action, caller | 외부 도달성과 보호 규칙 확인    | 유지/제외 판단    |
| 4. PoC 확인     | 대표 후보                                | emulator에서 일부 경로 확인 | 보조 근거       |
| 5. 개선안 도출     | 최종 이슈 유형                             | 통제 방안 정리            | PleOS 개선안   |

Agentic LLM으로 취약점 후보를 먼저 넓게 찾고, Android 검증으로 최종 보안 이슈 보고 대상을 좁혔다.

## 3.2 LLM 1차 후보 탐지

LLM은 최종 판정자가 아니라 보안 이슈 후보를 넓게 찾는 역할을 맡았다.

**47건**

보안 이슈 후보

**38건**

실제 이슈 포착

**9건**

보안 이슈 오탐

| 항목 | 내용                                                                                  |
|----|-------------------------------------------------------------------------------------|
| 입력 | Manifest 정보, decompiled code, 위험 API/context                                        |
| 역할 | 사람이 놓칠 수 있는 provider, receiver, transport, credential 타입의 취약점 후보를 전체 APK에 대해서 넓게 수집 |
| 한계 | Android 보안 사항을 충분히 고려하지 못한 취약점 오탐 9건                                                |

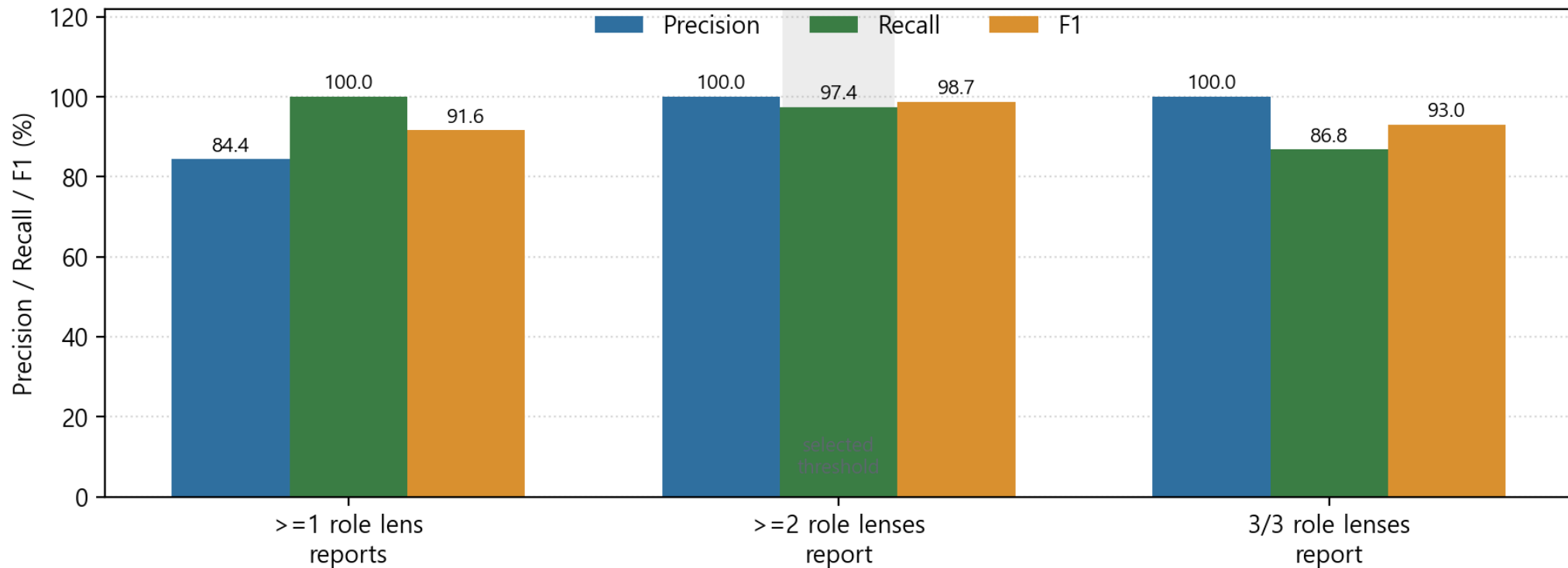
따라서, Agentic LLM이 찾은 취약점 후보 결과는 이슈로 바로 보고하면 안되고 공격 시나리오로 활용 가능한지 검증이 필요하다.

## 3.2 LLM 1차 후보 탐지

2개 이상 LLM role이 동의한 취약점 후보를 기준으로 오탐을 줄이면서 대부분의 실제 이슈를 유지한다.

### Role-Threshold Review: Precision-Recall Trade-Off

47 candidate findings evaluated with attacker, defender, and IVI-domain role prompts.



The >=1 threshold leaves 7 false positives; the 3/3 threshold misses 5 reference-positive candidates.  
The selected >=2 threshold removes all observed false-positive candidates while missing 1 low-severity reference-positive candidate.

## 3.3 Android 검증

코드상 위험한 API가 아니라 외부 앱이 실제로 도달 가능한지 여부로 최종 판단을 결정한다.

| 검증 항목      | 확인 질문                                          | 판단             |
|------------|------------------------------------------------|----------------|
| exported   | 외부 앱이 컴포넌트에 접근할 수 있는가?                         | 닫혀 있으면 제외 가능   |
| permission | signature-level 또는 명시 권한이 있는가?                 | 강한 권한이면 위험도 감소 |
| 호출 경로      | action, provider query, Binder call이 실제로 가능한가? | 경로가 없으면 제외     |
| 시스템 보호     | Android가 protected broadcast 등으로 막는가?          | 시스템 보호가 있으면 제외 |
| 실제 영향      | 데이터 노출, 상태 변화, 화면 영향이 있는가?                     | 영향이 있어야 보고 대상  |

## 3.4 최종 판단 기준

최종 이슈는 코드 위치, 외부 도달성, 권한 부재, 실제 영향이 함께 맞을 때만 남겼다.

| 번호 | 검증 질문              | 남기는 기준                              |
|----|--------------------|-------------------------------------|
| 1  | 컴포넌트가 외부에 열려 있는가?  | exported=true 또는 외부 호출 경로 존재        |
| 2  | 권한 제한이 충분한가?       | 권한이 없거나 일반 앱도 만족 가능                 |
| 3  | 일반 앱이 실제로 도달 가능한가? | query, broadcast, Binder call 경로 확인 |
| 4  | 보안상 결과가 있는가?       | 데이터 노출, 상태 변화, 로그/진단 전송 위험          |
| 5  | 개선안으로 연결 가능한가?     | 권한 추가, TLS 적용, validation 등 대응 가능   |

이 기준을 모두 만족하지 않으면, 위험 API가 있더라도 보안 이슈 최종 보고 대상에서 제외했다.

### 3.4 최종 판단 기준

주 결과와 보조 검증 트랙을 분리해 LLM 후보 판정의 해석 범위를 명확히 한다.

#### Supporting Validation Tracks And Their Measurement Roles

Each validation track answers a different question around the main static result; the metrics should not be pooled.

| Track                                | What it checks                              | Measured result                                                      | How to read it                       |
|--------------------------------------|---------------------------------------------|----------------------------------------------------------------------|--------------------------------------|
| Primary role-threshold adjudication  | Candidate finding report/reject quality     | TP 37, FP 0, FN 1<br>against 38-reference set;<br>F1 98.7%           | Main static result                   |
| Paired comparison                    | Report-all baseline vs final filtering rule | McNemar exact<br>p=0.0215                                            | Same-row improvement test            |
| Independent LLM cross-review         | Model/prompt robustness check               | 2-of-3 role-threshold F1:<br>86.6%;<br>best single-lens F1:<br>94.4% | Did not outperform selected workflow |
| Native-code boundary scan            | Selected native boundary sample             | 4 samples;<br>0 additional native-code-boundary vulnerabilities      | Boundary check, not F1               |
| Retrieval-grounded consistency check | Retrieval verdict and AAOS alignment        | Verdict agreement:<br>85.1%;<br>category agreement:<br>85.1%         | Intrinsic retrieval check            |

F1, paired-test p-values, native-boundary counts, and retrieval-alignment rates are intentionally reported separately.

## 4.1 평가 대상 47건

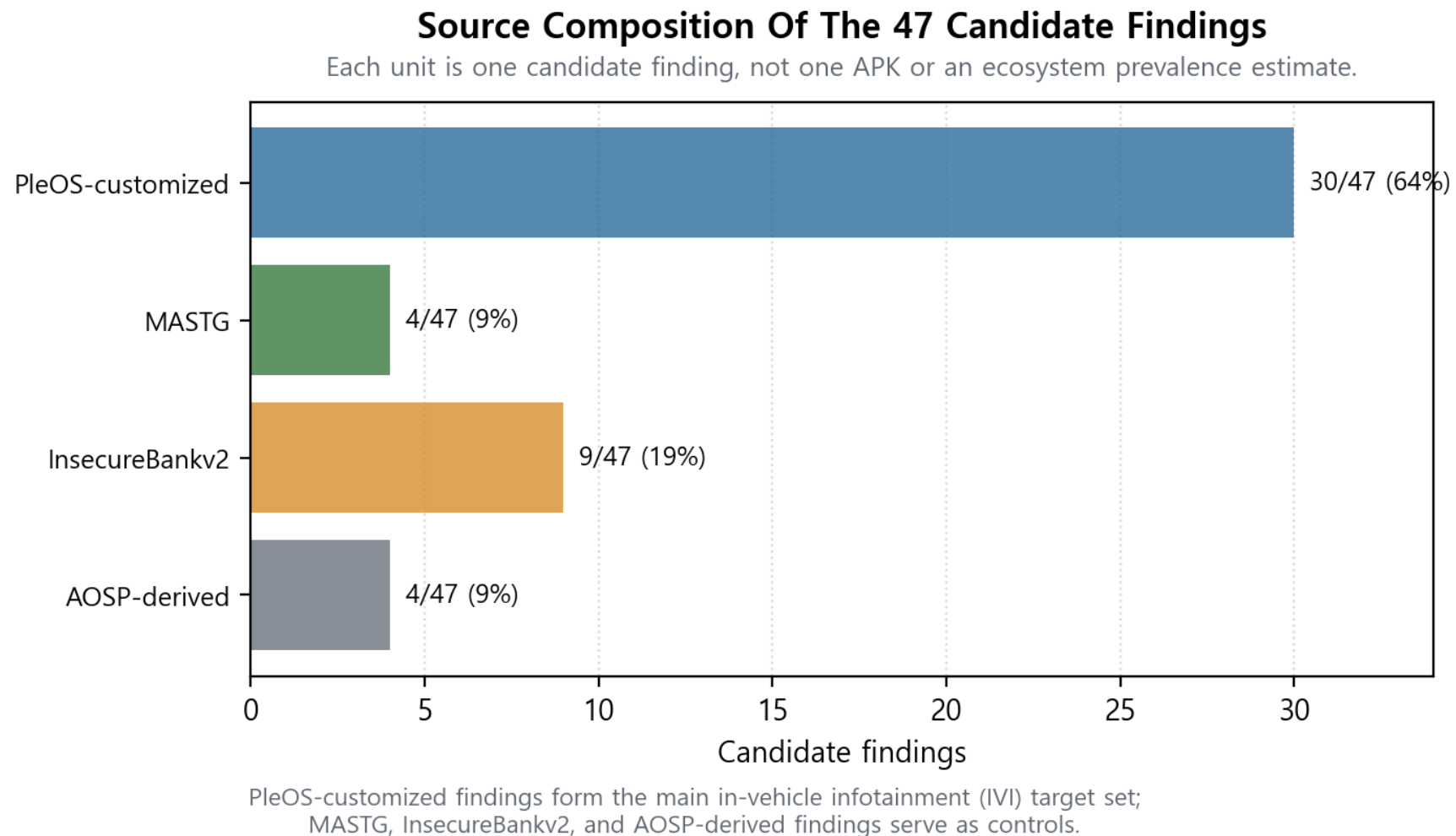
평가는 APK 개수가 아니라 보안 이슈 후보 47건을 기준으로 수행했다.



여기서 보안 이슈 1건은 provider, receiver, transport, credential 단위로 1개 APK에서 여러 이슈가 발생할 수 있다.

## 4.1 평가 대상 47건

평가는 APK 개수가 아니라 보안 이슈 후보 47건을 기준으로 수행했다.



## 4.2 1차 후보 탐지 결과

1차 후보 탐지는 실제 이슈 38건을 잡았지만 오탐 9건도 함께 포함했다.

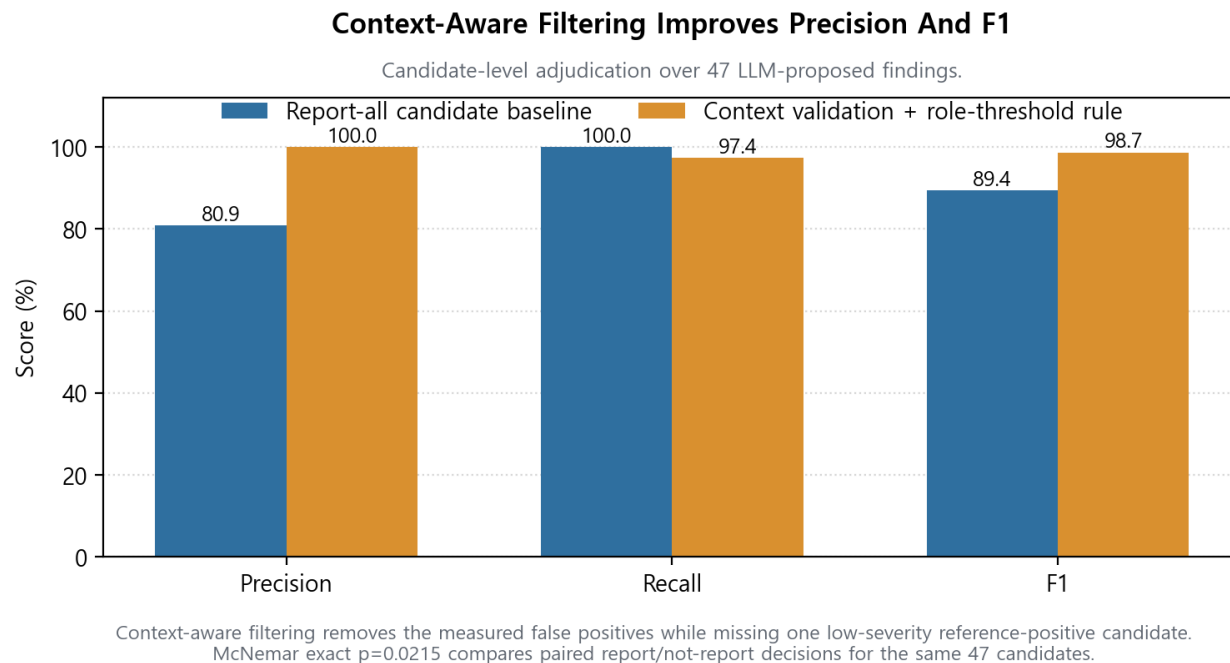
| 단계             | 결과                                                          |
|----------------|-------------------------------------------------------------|
| 보안 이슈 분석 단위 설계 | provider, receiver, transport, credential 등에 대해 보안 이슈 여부 판단 |
| LLM 1차 후보 탐지   | 실제 보안 이슈 47건 포착, 오탐 9건 포함                                   |
| Android 검증 후   | 오탐 9건 제거 / 미탐 1건 보고 (사람이 수동으로 찾았을 때 발견한 것으로 1건 이상 존재 가능)    |

**보안 이슈 후보 47건 → LLM 1차 후보 탐지 → Android 검증**

1차 탐지는 실제 이슈를 놓치지 않도록 넓게 잡고, 이후 검증 단계에서 오탐을 줄이는 방식으로 사용했다.

## 4.3 최종 검증 결과

Android 맥락 검증 후 오탐 9건을 제거했고 미탐 1건만 남았다.



| 구분       | 의미                                                            |
|----------|---------------------------------------------------------------|
| 오탐 9건 제거 | 위험한 코드를 포함하고 있지만, 호출 경로 및 Android 시스템 보호로 공격자의 접근이 차단된 취약점 후보 |
| 미탐 1건    | vc-7 (CarService.apk)                                         |
| 핵심 의미    | LLM 후보를 Android 검증으로 정제함                                      |

검증 단계는 9건의 잘못된 후보를 제거해 최종 보안 이슈 보고 목록을 정제했다.  
또한, 탐지하지 못한 취약점 1건에 대해서 아직은 보안 전문가의 추가 분석이 필요한 것으로 보인다.

## 4.4 오탐 제거 의미

위험 API 이름만으로는 부족하고 외부 도달성과 권한 확인이 최종 판단을 가른다.

---

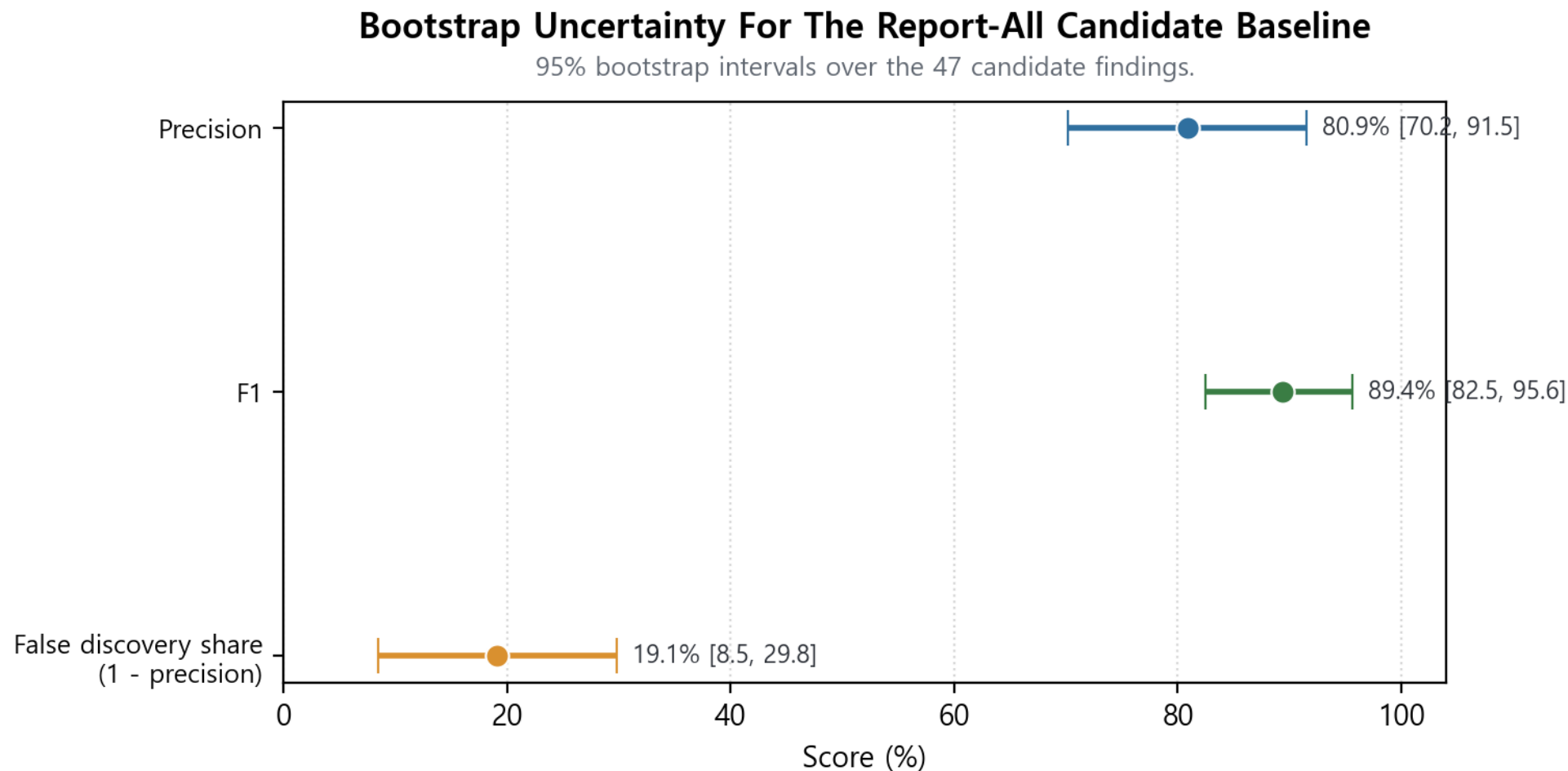
**위험 API가 코드에 있어도  
외부 앱이 도달할 수 없으면 최종 이슈가 아니다.**

Android permission, protected broadcast, framework check는 실제 공격 가능성을 바꾼다.

| 탐지하지 못한 취약점 | 탐지했으나 도달 불가능한 취약점 | 탐지하고 도달 가능한 취약점 |
|-------------|-------------------|-----------------|
| 1건          | 9건                | 38건             |

## 4.4 오탐 제거 의미

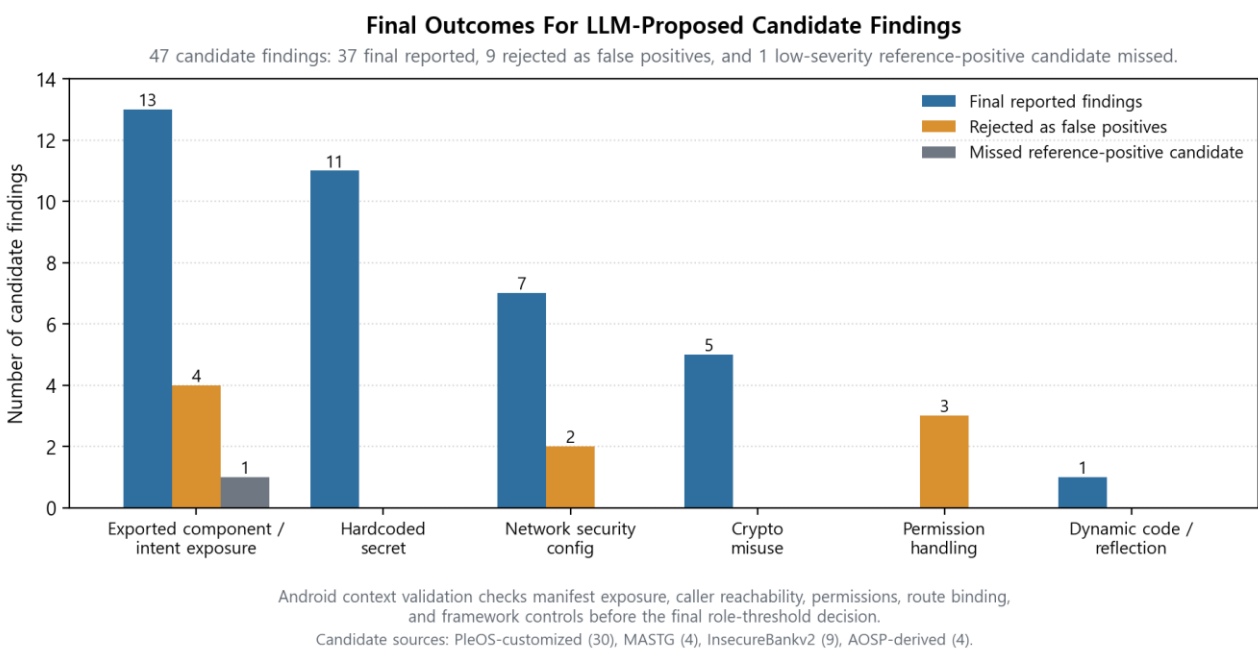
47건 기준의 정량 결과는 bootstrap interval과 함께 제한적으로 해석한다.



False discovery share is  $FP / (TP + FP)$ , shown as  $1 - \text{precision}$  to make the false-positive burden explicit.

# 4.5 보안 이슈 유형 요약

확인된 이슈는 provider, transport, receiver, credential 계열로 나뉜다.

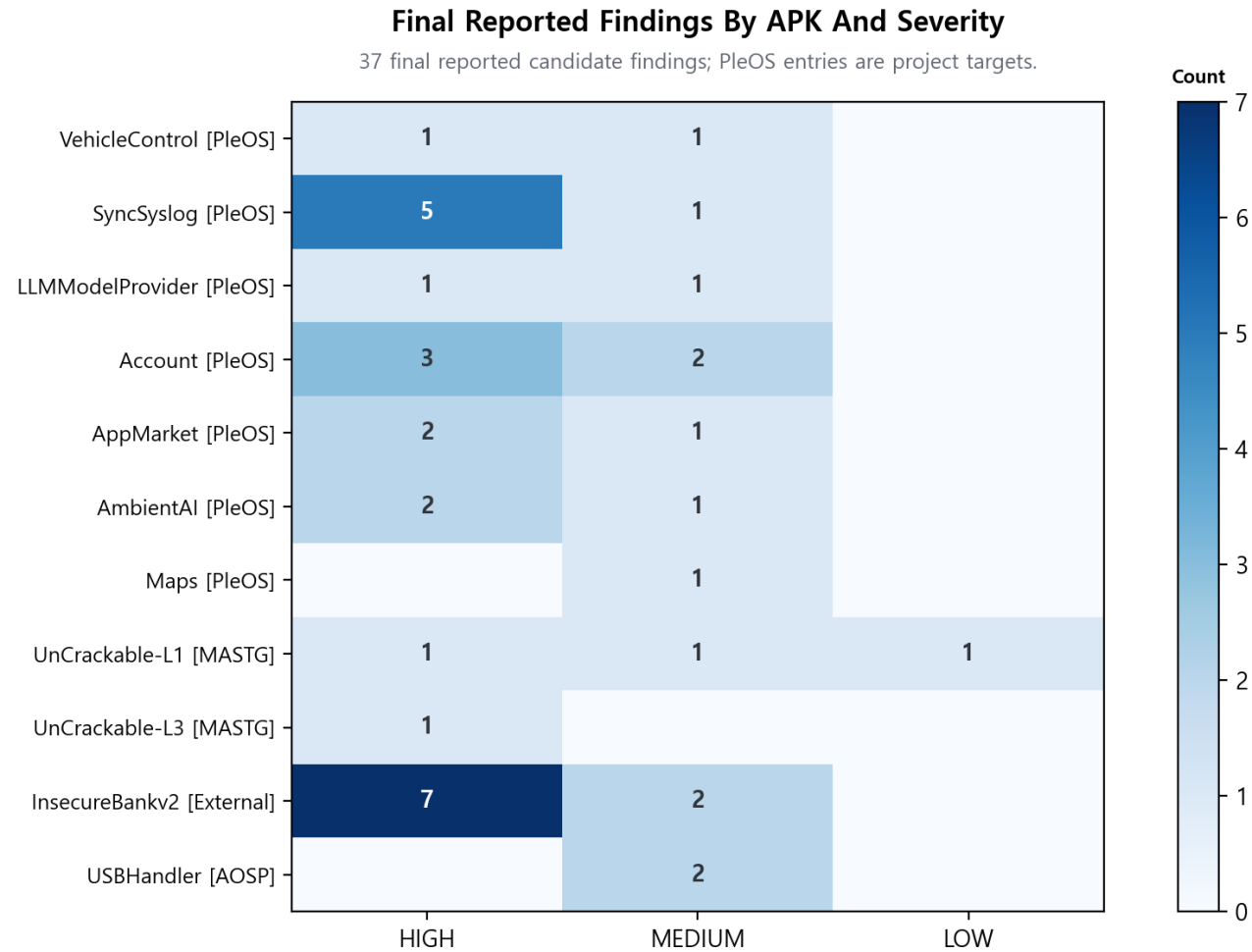


| 유형         | 예시                                  |
|------------|-------------------------------------|
| provider   | LLM prompt/corpus 접근 접점             |
| transport  | plaintext gRPC 사용                   |
| receiver   | payload 처리 입력 경로                    |
| credential | APK 내부 credential/secret pattern    |
| 오탐         | permission API, USB, BOOT_COMPLETED |

이후 Section 5에서 대표적인 provider, transport, receiver 사례와 오탐 제거 사례를 설명한다.

## 4.5 보안 이슈 유형 요약

최종 보고 후보를 APK와 severity 기준으로 나누어 후속 점검 우선순위를 정한다.



Use this as a follow-up priority map, not an ecosystem prevalence estimate.  
The one missed low-severity reference-positive candidate is excluded from final reported counts.

## 5.1 LLM prompt provider 노출

권한 없이 노출된 prompt provider는 민감 데이터 접근 접점이 될 수 있다.

```
<provider
  android:name=".PromptsContentProvider"
  android:authorities="ai.pleos.llm.prompts"
  android:exported="true"
  android:grantUriPermissions="true" />
```

query() -> prompt/corpus-like rows

| 항목     | 내용                                                               |
|--------|------------------------------------------------------------------|
| 확인 결과  | exported provider와 prompt/corpus query 구조를 확인했다.                 |
| 보안상 의미 | 권한 없이 접근 가능하면 prompt/corpus 데이터가 외부 앱에 노출될 수 있다.                 |
| 검증 내용  | manifest export 여부, read/write permission, query behavior를 확인했다. |

이 사례는 exported 상태와 권한 부재, query 동작이 함께 확인되어 보고 대상으로 남겼다.

## 5.2 provider 개선 방향

provider는 exported 설정, read/write 권한, caller 검증을 함께 점검해야 한다.

| 점검 항목         | 개선 방향                                   | 기대 효과            |
|---------------|-----------------------------------------|------------------|
| exported 설정   | 외부 접근이 필요 없으면 exported=false            | 불필요한 외부 query 차단 |
| read/write 권한 | signature-level permission 또는 명시적 권한 확인 | 허용된 앱만 접근        |
| caller 검증     | 허용된 package/UID만 query 허용               | 권한 우회 가능성 감소     |
| 데이터 최소화       | prompt/corpus 원문 대신 필요한 metadata만 제공    | 노출 시 피해 범위 축소    |

provider는 단일 설정보다 노출 여부, 권한, 호출자 검증, 제공 데이터 범위를 함께 줄이는 방향이 필요하다.

## 5.3 plaintext gRPC transport

plaintext gRPC는 진단·로그 전송 구간의 기밀성과 무결성 위험을 만든다.

```
ManagedChannelBuilder
    .forAddress(host, port)
    .usePlaintext()
    .build();

Context: diagnostic / system-log service
```

| 항목     | 내용                                                |
|--------|---------------------------------------------------|
| 확인 결과  | gRPC channel 설정에서 plaintext transport 사용을 확인했다.   |
| 보안상 의미 | diagnostic/log context의 전송 구간은 기밀성과 무결성 보호가 필요하다. |
| 검증 내용  | 단순 문자열이 아니라 실제 transport API 사용 여부를 확인했다.         |

단순히 plaintext 문자열이 있었던 것이 아니라, 실제 gRPC 채널 생성 과정에서 usePlaintext가 사용되는지 확인했다.

## 5.4 transport 개선 방향

transport 계열 이슈는 TLS/mTLS 적용과 cleartext 금지 정책으로 줄일 수 있다.

| 항목               | 권장 대응                              | 기대 효과            |
|------------------|------------------------------------|------------------|
| gRPC transport   | TLS 또는 mTLS 적용                     | 전송 구간 기밀성·무결성 보호 |
| cleartext policy | release build에서 cleartext 금지       | 운영 환경 평문 통신 방지   |
| diagnostic log   | 민감 로그 redaction 및 전송 대상 제한         | 로그 기반 정보 노출 축소   |
| 운영 확인            | endpoint, 인증서, debug/release 설정 분리 | 개발 설정의 운영 반영 방지  |

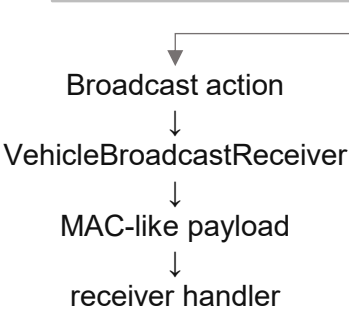
IVI 환경에서는 로그나 진단 데이터도 차량 상태나 사용자 정보와 연결될 수 있으므로 transport 보호가 필요하다.

# 5.5 VehicleBroadcastReceiver payload 처리

차량 주변 payload를 받는 receiver는 입력 검증과 권한 확인이 필요하다.

```
<receiver
  android:name=".VehicleBroadcastReceiver"
  android:exported="true">
  <intent-filter> ... </intent-filter>
</receiver>
```

| 항목     | 내용                                                  |
|--------|-----------------------------------------------------|
| 확인 결과  | vehicle-adjacent receiver와 payload 처리 경로를 확인했다.     |
| 보안상 의미 | 외부 입력이 차량 주변 상태 처리 로직으로 이어질 수 있다.                   |
| 검증 내용  | export 여부, action, permission, payload shape를 확인했다. |



receiver가 외부 입력을 받을 수 있는지, 그 입력이 실제 handler까지 이어지는지를 중심으로 검증했다.

## 5.6 receiver 개선 방향

receiver는 signature 권한, action 제한, payload validation을 적용해야 한다.

| 점검 항목              | 개선 방향                                  | 기대 효과                |
|--------------------|----------------------------------------|----------------------|
| receiver export    | 외부 호출이 필요 없으면 export 제한                | 불필요한 broadcast 입력 차단 |
| action 제한          | 허용 action만 처리                          | 의도하지 않은 action 처리 방지 |
| 권한 확인              | signature permission 또는 caller 검증      | 일반 앱의 임의 호출 제한       |
| payload validation | MAC/address-like payload 형식과 source 검증 | 비정상 입력으로 인한 오동작 방지   |

receiver는 외부 입력을 받는 구조이기 때문에, 입력 경로와 payload 자체를 모두 검증해야 한다.

## 5.7 오탐 제거 사례 1

permission API가 코드에 있어도 외부 호출 경로가 막혀 있으면 최종 이슈가 아니다.

---

| 단계      | 내용                                               |
|---------|--------------------------------------------------|
| 처음 보인 것 | permission grant/revoke API                      |
| 검증한 것   | manifest / navigation / deep link / caller route |
| 최종 판단   | 외부 호출 경로가 없어 제외                                  |

**permission grant/revoke API → 호출 경로 검증 → 외부 도달 불가로 제외**

위험 API가 있어도 일반 앱이 도달할 수 없으면 최종 보안 이슈로 보고하지 않았다.

## 5.8 오탐 제거 사례 2

USB와 BOOT\_COMPLETED 사례는 Android 시스템 보호 규칙 확인의 필요성을 보여준다.

---

| 사례    | 처음 위험해 보인 이유                          | 제외한 이유                                            |
|-------|---------------------------------------|---------------------------------------------------|
| usb-2 | USB permission path가 외부 트리거 가능해 보였음   | framework permission check로 normal app trigger 차단 |
| ss-1  | exported BOOT_COMPLETED receiver로 보였음 | BOOT_COMPLETED는 일반 앱이 임의로 보낼 수 없는 시스템 브로드캐스트      |

**Android 시스템 보호 규칙을 확인하지 않으면 실제로는 막힌 경로를 취약점으로 오판할 수 있다.**

이 사례는 Android 플랫폼 지식이 보안 판단에 직접 영향을 준다는 점을 보여준다.

## 5.9 대표 사례 결론

대표 사례의 결론은 API 이름이 아니라 도달성과 권한이 판단 기준이라는 점이다.

| 보고 대상으로 남긴 사례                          | 최종 제외한 사례              |
|----------------------------------------|------------------------|
| 외부 접점이 확인됨                             | 위험 API 이름만 확인됨         |
| 권한 부재 또는 제한 부족                         | 호출 경로가 없음              |
| query, broadcast, transport 등 도달 경로 확인 | Android 시스템 보호 규칙으로 차단 |
| 데이터 노출, 상태 변화, 전송 보호 문제 등 영향 존재        | 실제 보안 결과가 확인되지 않음      |

최종 판단 기준은 “위험해 보이는 코드”가 아니라 “외부에서 도달 가능하고 보안 영향이 있는 코드”다.

## 6.1 PoC 목적과 환경

PoC는 일부 후보가 에뮬레이터 환경에서 실제로 호출 가능한지 확인한 보조 근거다.

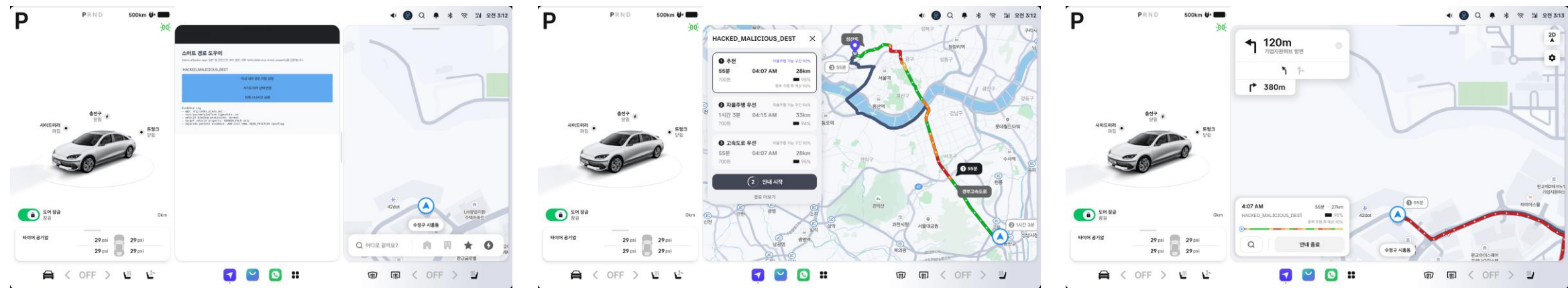
---

| 구분    | 내용                                                          |
|-------|-------------------------------------------------------------|
| 목적    | 정적분석 후보 중 일부가 에뮬레이터에서 호출 가능한지 확인                            |
| 환경    | 같은 IVI 환경에 설치된 검증 앱 + PleOS/AAOS emulator                   |
| 확인 대상 | route/property, provider query, receiver delivery, 권한 차단 경로 |
| 역할    | 정량 평가를 바꾸는 실험이 아니라 대표 이슈의 보조 근거                             |

PoC는 정적분석 결과가 실제 호출 가능성과 연결되는지 제한적으로 확인하기 위한 과정이다.

# 6.2 route/property PoC

검증 앱으로 내비게이션 화면 접점과 MIRROR\_FOLD 상태 값 접점을 각각 확인했다.

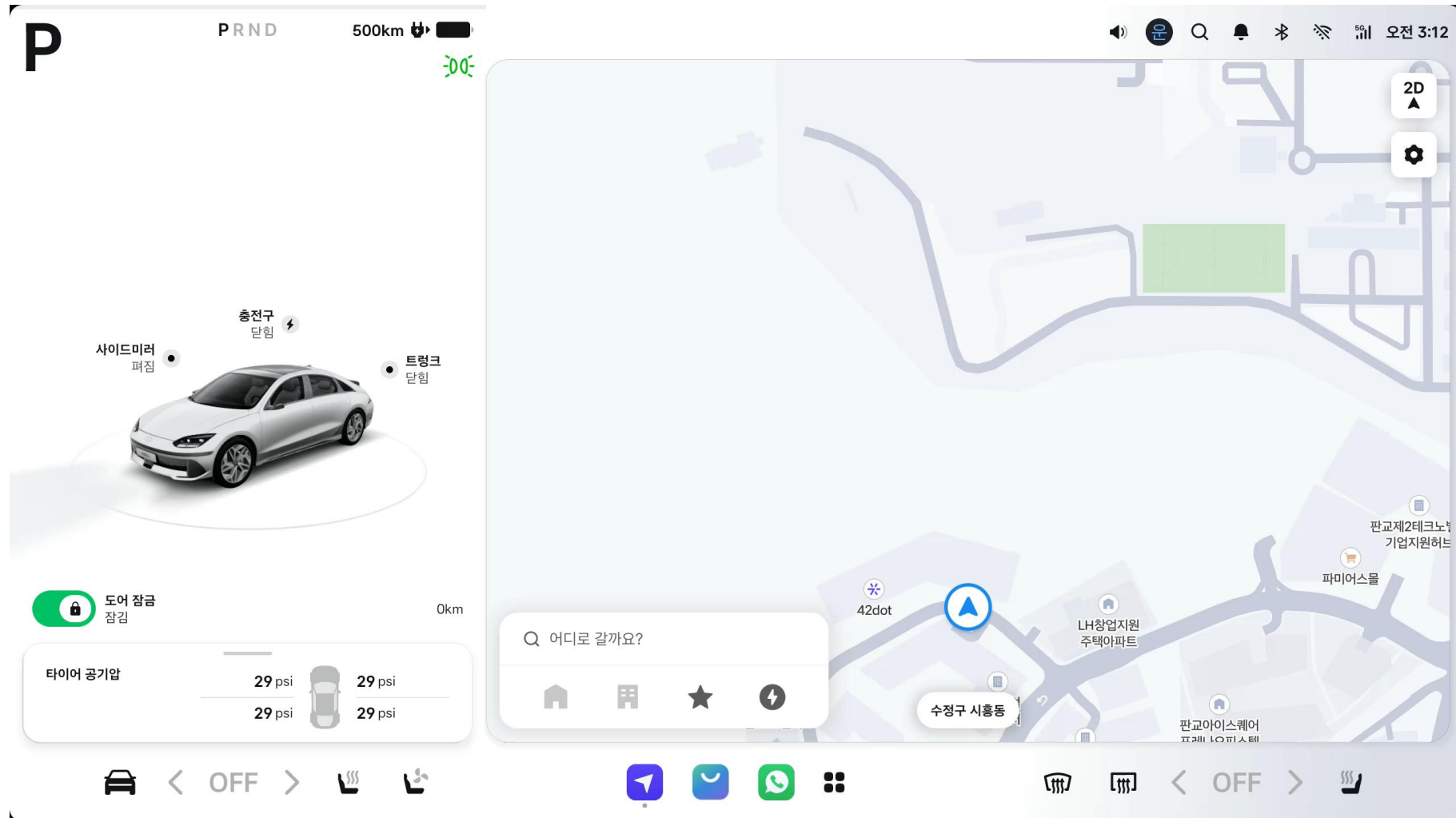


| 확인 대상        | 확인 내용                            |
|--------------|----------------------------------|
| Route PoC    | IVI 내비게이션 화면 접점에 대한 영향을 확인       |
| Property PoC | 되돌릴 수 있는 MIRROR_FOLD 상태 값 접점을 확인 |

이 PoC는 active route hijack이나 실제 차량 제어가 아니라, 에뮬레이터에서 특정 route/property 접점의 접근 가능성을 확인한 것이다.

## 6.3 route/property PoC 영상 근거

route/property PoC는 영상으로 설명한다.



## 6.4 provider query / receiver delivery

provider query와 receiver delivery도 제한적으로 접근 가능성을 확인했다.

---

Prompt Leak -> Prompt-Aware Input PoC

Goal

### What this proves

The provider leak is useful because it reveals the IVI LLM scenario and template contract. An attacker can then build inputs that fit that contract better than generic text.

**Not a live jailbreak, not vehicle actuation, not backend abuse.**

## 6.5 권한 차단 확인 사례

권한 확인으로 막히는 경로는 최종 보안 이슈에서 제외했다.

---

**PairedDevices provider**

**READ / WRITE 권한 확인**

**검증 앱 query → PairedDevices provider → READ / WRITE permission check → 접근 차단 확인**

접근 가능한 사례만 본 것이 아니라, 접근이 막히는 사례도 확인했다. 권한 확인이 실제로 동작하면 최종 보안 이슈에서 제외한다.

## 6.6 PoC 주장 범위

PoC 결과는 실차 제어, 원격 공격, 자율주행 탈취 성공을 의미하지 않는다.

---

| 확인한 것                        | 주장하지 않는 것                                    |
|------------------------------|----------------------------------------------|
| route preview UI 영향          | active route hijack                          |
| 되돌릴 수 있는 MIRROR_FOLD 상태 값 변경 | steering / brake / gear / powertrain control |
| provider query 접근 가능성        | remote vehicle takeover                      |
| receiver delivery 접근 가능성     | real-vehicle actuation                       |

PoC는 에뮬레이터 환경에서 일부 경로의 접근 가능성을 확인한 보조 근거로 제한한다.

## 7.1 PleOS 적용 개선안

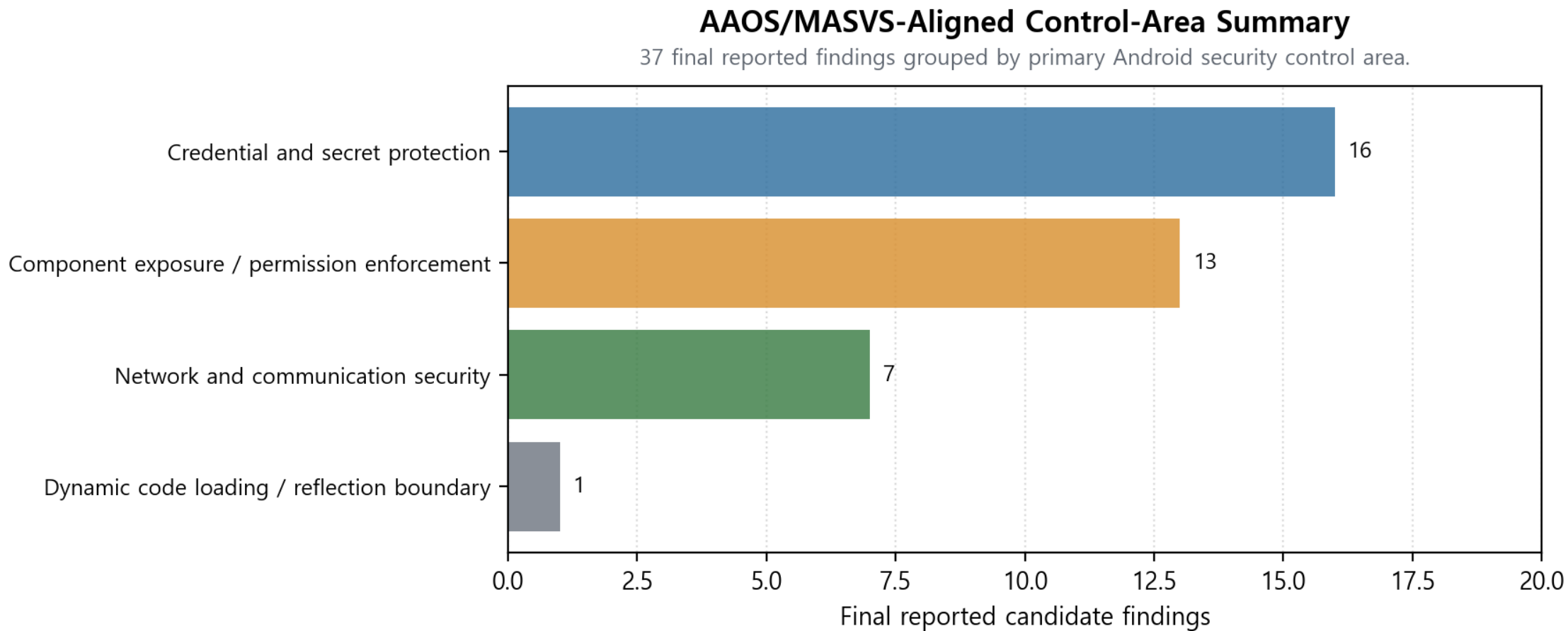
점검 결과는 PleOS에 적용할 보안 통제 목록으로 연결된다.

| 이슈 유형                  | 적용할 보안 통제                                                |
|------------------------|----------------------------------------------------------|
| hardcoded credential   | credential rotation, short-lived token, client secret 제거 |
| exported provider      | exported=false 또는 signature-level read/write 권한          |
| plaintext gRPC         | TLS/mTLS 적용, cleartext 금지                                |
| exported receiver      | receiver validation, action 제한, caller 확인                |
| Binder / property path | caller UID 확인, property whitelist, audit logging         |

최종 목표는 문제 나열이 아니라 PleOS 환경에서 적용 가능한 보안 통제로 연결하는 것이다.

## 7.1 PleOS 적용 개선안

최종 보고 후보는 credential 보호, component 권한, network 보안 통제에 주로 집중된다.



This maps the final reported set, not AAOS prevalence; the missed low-severity reference-positive candidate is excluded.

## 7.2 우선 적용 권장 항목

가장 먼저 적용할 항목은 권한 확인, TLS/mTLS, receiver 검증, Binder 호출자 확인이다.

---

| 우선순위 | 항목               | 이유                                                     |
|------|------------------|--------------------------------------------------------|
| 1    | Provider 권한 확인   | 외부 query가 필요한 경우에도 read/write 권한과 caller 검증을 적용한다.     |
| 2    | TLS/mTLS 전환      | 진단·로그 전송 구간에서 plaintext 사용을 제거한다.                      |
| 3    | Receiver 입력 검증   | action, caller, payload 형식을 함께 검증한다.                   |
| 4    | Binder caller 확인 | UID/package 기반 caller check와 property whitelist를 적용한다. |

먼저 외부 접점의 권한 확인과 cleartext 제거를 적용하면 가장 큰 위험을 줄일 수 있다.

## 7.3 공개 가능 자료와 비공개 원본

민감 원본은 비공개로 두고 공개 가능한 요약 근거만 분리한다.

<https://github.com/HoyoenKim/pleos-llm-scanner>

| 공개 가능                 | 비공개 유지                        |
|-----------------------|-------------------------------|
| 보안 이슈 유형, 수치, 개선안     | raw APK                       |
| redacted code excerpt | JADX output                   |
| 요약 chart와 발표용 근거      | raw logs, prompt rows, tokens |
| 비식별화된 PoC 설명          | 민감 screenshot/video 원본        |

공개 자료는 연구 결과와 개선 방향을 설명하는 수준으로 제한하고, 재현에 필요한 민감 원자료는 비공개로 유지한다.

## 7.4 연구 한계

결과는 측정한 47건과 에뮬레이터 PoC 범위 안에서 해석해야 한다.

---

| 한계     | 의미                                 |
|--------|------------------------------------|
| 표본 한계  | 47건 측정 결과이며 전체 앱스토어 기준 성능은 아님      |
| 라벨 한계  | PleOS 항목은 내부 기준으로 정리했고 독립 검토가 더 필요 |
| PoC 한계 | 에뮬레이터 환경의 제한적 재현이며 실차 공격이 아님       |
| 분석 한계  | native/JNI/runtime 흐름은 추가 분석 필요    |

결과는 전체 Android 생태계 성능이 아니라, 본 연구에서 측정한 PleOS IVI APK 후보와 검증 환경 기준으로 해석해야 한다.

## 7.5 최종 결론

핵심은 LLM 단독 탐지가 아니라 검증 가능한 PleOS IVI APK 보안 점검 절차다.

---

**PleOS IVI APK의 provider, receiver, transport, credential 등 주요 외부 접점을 점검했다.**

LLM은 보안 이슈 후보를 넓게 찾는 데 사용했고, Android 권한·호출 경로·시스템 보호 규칙 검증으로 오탐 9건을 제거했다.

대표 이슈는 PoC 근거와 개선안으로 연결했으며, 최종 기여는 LLM 후보 탐지와 Android 맥락 검증을 결합한 보안 점검 절차다.

**LLM 후보 탐지 + Android 맥락 검증 = 보고 가능한 보안 이슈 선별**